

Advanced Computing Project

Projeto em Computação Avançada (MEI/MCA)

Tutorial 1: How To Write a Scientific Paper?

University of Minho
2025-2026



Why writing matters

Implementation $\neq$ contribution

- ❖ A system without a paper
 - Is invisible to the community (there are exceptions of course)
 - Cannot be evaluated, reused, or built upon
- ❖ Results alone are not enough
 - Numbers do not explain why they matter or which problem they address
 - Without context, they are just benchmarks

Organization

Typical organization of a systems paper

- ❖ What is the paper about?
 - Title, Abstract
- ❖ What is the main problem it addresses?
 - Introduction, Background, Problem analysis/Motivation
- ❖ How does the paper overcome this problem?
 - Design, Implementation
- ❖ How does the proposed solution compare with prior work?
 - Evaluation, Related work
- ❖ What are the takeaways?
 - Conclusion

Organization

Typical organization of a systems paper

- ❖ What is the paper about?

- Title, Abstract

This is not arbitrary, it tells a story

Problem → Insight → Solution/System → Evidence

- Design, Implementation

- ❖ How does the proposed solution compare with prior work?

- Evaluation, Related work

- ❖ What are the takeaways?

- Conclusion

Rolling example

❖ SplitFS: Reducing Software Overhead in File Systems for Persistent Memory

- ▶ 27th ACM Symposium on Operating Systems Principles (SOSP'19)
- ▶ **URL:** <https://dl.acm.org/doi/pdf/10.1145/3341301.3359631>
- ▶ **Code:** <https://github.com/utsaslab/SplitFS>



SplitFS: Reducing Software Overhead in File Systems for Persistent Memory

Rohan Kadekodi
University of Texas at Austin

Se Kwon Lee
University of Texas at Austin

Sanidhya Kashyap
Georgia Institute of Technology

Taesoo Kim
Georgia Institute of Technology

Aasheesh Kolli
Pennsylvania State University and
VMware Research

Vijay Chidambaram
University of Texas at Austin and
VMware Research

Abstract

We present SPLITFS, a file system for persistent memory (PM) that reduces software overhead significantly compared to state-of-the-art PM file systems. SPLITFS presents a novel split of responsibilities between a user-space library file system and an existing kernel PM file system. The user-space library file system handles data operations by intercepting POSIX calls, memory-mapping the underlying file, and serving the read and overwrites using processor loads and stores. Metadata operations are handled by the kernel PM file system (ext4 DAX). SPLITFS introduces a new primitive termed relink to efficiently support file appends and atomic data operations. SPLITFS provides three consistency modes, which different applications can choose from, without interfering with each other. SPLITFS reduces software overhead by up-to 4× compared to the NOVA PM file system, and 17× compared to ext4 DAX. On a number of micro-benchmarks and applications such as the LevelDB key-value store running the YCSB benchmark, SPLITFS increases application performance by up to 2× compared to ext4 DAX and NOVA while providing similar consistency guarantees.

CCS Concepts • Information systems → Storage class memory; • Hardware → Non-volatile memory; • Software and its engineering → File systems management;

Keywords Persistent Memory, File Systems, Crash Consistency, Direct Access

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SOSP '19, October 27–30, 2019, Huntsville, ON, Canada

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6873-5/19/10...\$15.00

<https://doi.org/10.1145/3341301.3359631>

ACM Reference Format:

Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. 2019. SplitFS: Reducing Software Overhead in File Systems for Persistent Memory. In *SOSP '19: Symposium on Operating Systems Principles, October 27–30, 2019, Huntsville, ON, Canada*. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3341301.3359631>

1 Introduction

Persistent Memory (PM) is a new memory technology that was recently introduced by Intel [17]. PM will be placed on the memory bus like DRAM and will be accessed via processor loads and stores. PM has a unique performance profile: compared to DRAM, loads have 2–3.7× higher latency and $1/3^{rd}$ bandwidth, while stores have the same latency but $1/6^{th}$ bandwidth [18]. A single machine can be equipped with up to 6 TB of PM. Given its large capacity and low latency, an important use case for PM will be acting as storage.

Traditional file systems add large overheads to each file-system operation, especially on the write path. The overhead comes from performing expensive operations on the critical path, including allocation, logging, and updating multiple complex structures. The systems community has proposed different architectures to reduce overhead. BPFS [7], PMFS [24], and NOVA [33] redesign the in-kernel file system from scratch to reduce overhead for file-system operations. Aerie [28] advocates a user-space library file system coupled with a slim kernel component that does coarse-grained allocations. Strata [19] proposes keeping the file system entirely in user-space, dividing the system between a user-space library file system and a user-space metadata server. Aerie and Strata both seek to reduce overhead by not involving the kernel for most file-system operations.

Despite these efforts, file-system data operations, especially writes, have significant overhead. For example, consider the common operation of appending 4K blocks to a file (total 128 MB). It takes 671 ns to write a 4 KB to PM; thus, if performing the append operation took a total of 675 ns,

Rohan Kadekodi and Se Kwon Lee are supported by SOSP 2019 student travel scholarships from the National Science Foundation.

Title

❖ The **title** must

- capture the core idea in one sentence
- be clear, specific, and informative
- often includes the problem and the approach
- **Goal:** make readers want to read the paper!

❖ Examples

- *“Are you sure you want to use mmap on your database management system?”*

❖ Alternative titles (https://www.youtube.com/watch?v=1BRGU_AS25c)

- *“MMAP will ruin your DBMS and your life”*
- *“Read a F***ing book and don’t use MMAP in your DBMS”*
- *“MMAP ==  ”*

Abstract

- ❖ **Purpose:** A compressed version of the entire paper
- ❖ Must include
 - Problem
 - Key insight
 - Proposed solution
 - Main results
- ❖ Why does it matter?
 - Many readers only read the abstract
 - Reviewers form first impressions here
 - Determines whether the paper gets attention

Introduction

- ❖ The introduction is a **structured argument**, not just context
- ❖ Context (fade-in) & importance
 - Why does this area matter?
- ❖ Problem (statement)
 - What is broken or inefficient (or even stupid?)
- ❖ Limitations of existing work
 - Why are current solutions insufficient?
 - Can be “explained” or “shown”

Introduction

❖ Key insight

- The central idea that unlocks the solution (💡)

❖ Proposed solution (high-level)

- What have you built to overcome this problem?
- Which techniques/mechanisms have you designed?

❖ Experimental results

- How good is your solution compared to prior work?

❖ Contributions

- Clear, concrete summary of the main contributions of the paper

Introduction

❖ Key insight

- The central idea that unlocks the solution (💡)

A strong introduction makes the reader think:

“This problem is important, and this solution makes sense”

❖ Experimental results

- How good is your solution compared to prior work?

❖ Contributions

- Clear, concrete summary of the main contributions of the paper

Background

- ❖ **Purpose:** Provide the **minimum knowledge** needed to understand the paper
- ❖ Introduce key concepts, system models, terminology, etc.
- ❖ What to include?
 - Only what is strictly necessary
 - Brief explanation of how existing systems work and relevant abstractions
- ❖ What to avoid?
 - Long textbook explanations
 - Irrelevant details
 - Overloading the reader

Motivation

❖ The **motivation justifies the paper**

- Show that the problem is real, important, and non-trivial
- Provide evidence, not just claims (*“show, don’t tell”*)

❖ What should it contain?

- Empirical observations (experiments, measurements)
- Concrete examples of inefficiencies
- Analysis of *why* current approaches fail
- **Takeaways!!!**

❖ Why does it matter?

- Without it, the paper feels unmotivated
- *“Earn the right”* to propose a solution

Design

- ❖ **Purpose:** Translate the key insight (and takeaways from the motivation) into a concrete system
- ❖ Explain how the solution works
- ❖ Typical structure
 - **High-level overview**, with the architecture diagram and main idea of the system
 - **Core components** that drive the design of the system (key modules/building blocks)
 - **Techniques & mechanisms**, including algorithms, policies, and data structures
- ❖ Best practices
 - Use diagrams to explain the architecture
 - Keep a clear mapping between the problem and design decisions
 - Avoid unnecessary low-level detail

Design

- ❖ **Purpose:** Translate the key insight (and takeaways from the motivation) into a concrete system
- ❖ Explain how the solution works

The design should feel like a natural consequence of the insight

- **Core components** that drive the design of the system (key modules/building blocks)
- **Techniques & mechanisms**, including algorithms, policies, and data structures
- ❖ Best practices
 - Use diagrams to explain the architecture
 - Keep a clear mapping between the problem and design decisions
 - Avoid unnecessary low-level detail

Implementation

- ❖ **Purpose:** Show that the system is real and practical
- ❖ What to include
 - **High-level implementation** details, such as key technologies or frameworks used, **architecture aspects** such as user-space/kernel-space, libraries, etc.
 - **Scope of the implementation:** code size, complexity, changes to existing systems
- ❖ What to avoid
 - Line-by-line code explanations
 - Excessive low-level detail

Implementation

❖ **Purpose:** Show that the system is real and practical

The implementation shows the design is not just an idea, and actually works.

▸ **Scope of the implementation:** code size, complexity, changes to existing systems

❖ What to avoid

- Line-by-line code explanations
- Excessive low-level detail

Experimental evaluation

- ❖ **Purpose:** demonstrate that the system works correctly and improves over existing solutions (provide convincing evidence!)
- ❖ What to include
 - **Clear evaluation questions:** What are we trying to prove?
 - **Experimental setup:** hardware, software, workloads, configurations, metrics
 - **Baselines:** Which existing systems are we comparing against?
- ❖ Best practices
 - Explain **why** results happen (not just show them or state the obvious)
 - Include trade-offs and limitations
 - Use plots with clear messages (avoid redundancy)

Related work

- ❖ **Purpose:** show how your work fits into the existing literature, and highlight what is new and different
- ❖ What to include
 - Key prior systems and approaches
 - Grouped by categories
 - Brief explanation of what they do and their limitations
- ❖ What to avoid
 - Long lists of papers without comparison
 - Describing work that is not directly relevant
 - Omitting related work

Conclusion

- ❖ **Purpose:** Summarize the core message of the paper, by reinforcing the problem, insight, and contributions
- ❖ What to include
 - Brief recap of the problem and why it matters, key idea/approach, and main results
- ❖ What to avoid
 - Repeating the entire paper
 - Introducing new ideas or results

Add-ons

Optional (but valuable) sections

❖ Discussion and limitations

- Insights beyond the core design
- Lessons learned during development
- When the system does not perform well
- Trade-offs and assumptions

❖ Future work

- Open problems and future directions

❖ Appendix/Extra material

- Additional experiments, proofs, etc.

Questions?